COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	2026-09-03	**DATE SUBMITTED:**	2026-09-05
NAME:	Ranzel Aldous Gabriel L. Dueñas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6 **Targeting Specific Nodes**

1. PROCEDURE OUTPUT

Task 1: Targeting Specific Nodes

1. Create a new yaml file called “site.yaml”

```
---

- hosts: all
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
~
~
```

2. Edit site.yaml

```
---

- hosts: all
  become: true
  pre_tasks:

    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cach: yes
      when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"


- hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
```

- result

```
PLAY RECAP *********************************************************
********
centos1                    : ok=2    changed=0    unreachable=0    faile
d=0    skipped=1    rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    faile
d=0    skipped=2    rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    faile
d=0    skipped=2    rescued=0    ignored=0

localserver@workstation:~/CPE232_Duenas/ansible-lab$
```

3. Edit site.yaml, add plays

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      yum:
        name: mariadb_server
        state: latest
      when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true
    - name: install mariadb package (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "Ubuntu"
```

- result

```
PLAY RECAP *********************************************************************************
centos1                    : ok=5    changed=2    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0

localserver@workstation:~/CPE232_Duenas/ansible-lab$
```

4. Run systemctl status mariadb on db_server and web_server.

```
[centos1@localhost ~]$ systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
     Active: active (running) since Thu 2026-09-03 10:01:08 PST; 4min 53s ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 12148 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
    Process: 12170 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
    Process: 12269 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
   Main PID: 12253 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 8 (limit: 10500)
     Memory: 75.7M (peak: 100.5M)
        CPU: 699ms
     CGroup: /system.slice/mariadb.service
             └─12253 /usr/libexec/mariadbd --basedir=/usr

Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: The second is mysql@localhost, it has no password either, but
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: you need to be the system 'mysql' user to connect.
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: After connecting you can set the password, if you would need to be
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: able to connect as any of these users with a password and without sudo
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: Please report any problems at https://mariadb.org/jira
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: The latest information about MariaDB is available at https://mariadb.org/.
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: Consider joining MariaDB's strong and vibrant community:
Sep 03 10:01:08 localhost.localdomain mariadb-prepare-db-dir[12209]: https://mariadb.org/get-involved/
Sep 03 10:01:08 localhost.localdomain systemd[1]: Started MariaDB 10.5 database server.
[centos1@localhost ~]$
```

- This happened because we configured the db_servers to be centos. That's why it is able to check the status since it installs mariadb there

```
ubuntu2@server2:~$ systemctl status mariadb
Unit mariadb.service could not be found.
ubuntu2@server2:~$
```

```
ubuntu1@server1:~$ systemctl status mariadb
Unit mariadb.service could not be found.
ubuntu1@server1:~$
```

- This happened because the configured db_servers doesn't include ubuntu, only centos.

5. Edit site.yaml and configure installation file_servers group.

```
- hosts: file_servers
  become: true
  tasks:

    - name: install samba package
      package:
        name: samba
        state: latest
```

- result

```
PLAY RECAP *********************************************************************************
centos1                    : ok=5    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0

localserver@workstation:~/CPE232_Duenas/ansible-lab$
```

- This play targets all servers under the “file_servers” group.

Task 2: Using Tags in running playbooks

1. Edit site.yaml

```
---

- hosts: all
  become: true
  pre_tasks:

    - name: install updates (CentOS)
      tags: always
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      tags: always
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"
```

```
- hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      tags: apache,apache2,ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      tags: apache,centos,httpd
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      tags: centos, db, mariadb
      dnf:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true
    - name: install mariadb package (Ubuntu)
      tags: db, mariadb,ubuntu
      apt:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "Ubuntu"
```

```
- hosts: file_servers
  become: true
  tasks:

    - name: install samba package
      tags: samba
      package:
        name: samba
        state: latest
```

- ansible-playbook –ask-become-pass site.yaml

```
PLAY RECAP *********************************************************************
centos1                    : ok=5    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
```

- Since we changed, each tag runs first. And then the remaining task will be either executed or skipped

2. Local Machine
   ansible-playbook –list-tags site.yaml

```
playbook: site.yaml

  play #1 (all): all    TAGS: []
      TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
      TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

  play #3 (db_servers): db_servers      TAGS: []
      TASK TAGS: [centos, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers  TAGS: []
      TASK TAGS: [samba]
```

- It lists all available tags organized by play.

ansible-playbook –tags centos –ask-become-pass site.yaml

```
PLAY RECAP *********************************************************************
centos1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=3    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
```

- It will only execute tasks with centos tags, along with any tasks tagged “always”.

ansible-playbook –tags db –ask-become-pass site.yaml

```
PLAY RECAP *********************************************************************
centos1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
ubuntu2                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
```

- Executes tasks tagged “db” along side “always, skipping untagged tasks such as MariaDB service restart.

ansible-playbook –tags apache –ask-become-pass site.yaml

```
PLAY RECAP *********************************************************************
centos1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
```

- Executes tasks tagged “apache” alongside “always”. It bypasses all database and file server plays entirely.

ansible-playbook –tags “apache,db” –ask-become-pass site.yaml

```
PLAY RECAP *********************************************************************
centos1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
```

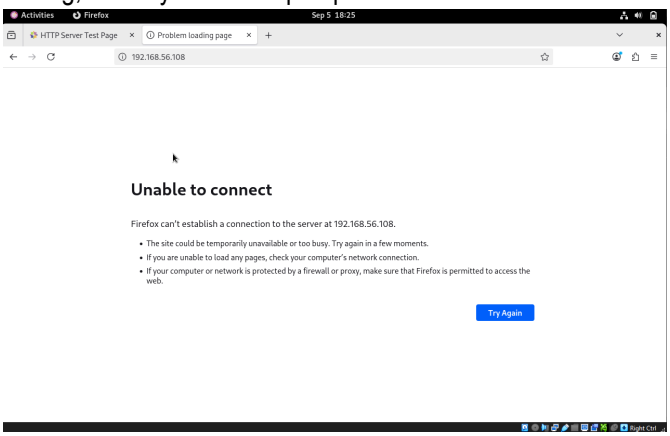
- Executes tasks tagged “apache” or “db” alongside with “always”.

Task 3: Managing Services

1. Edit site.yaml

```
- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

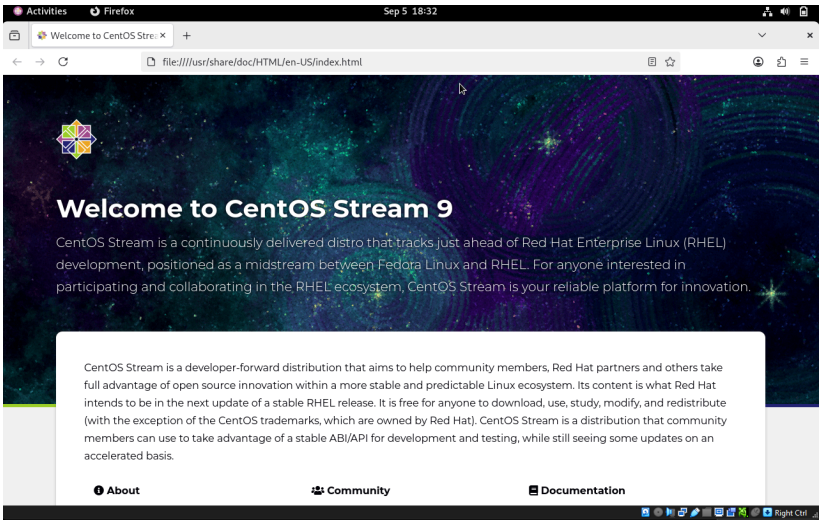
2. Testing; sudo systemctl stop httpd. Browse server IP.



3. Local machine; run site.yaml

```
PLAY RECAP *********************************************************************************
centos1                    : ok=5    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=3
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=3
rescued=0    ignored=0
```

Browse server IP on centos



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Creating a Role for Web Servers
2. Creating a Role for Web Servers
3. Creating a Role for Database Servers
4. Combining Multiple Roles in a Playbook

```
---
- name: System Pre-Tasks (Updates)
  hosts: all
  become: true
  pre_tasks:

    - name: install updates (CentOS)
      tags: always
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      tags: always
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

- name: Configure Web Servers
  hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      tags: [apache, apache2, ubuntu, web]
```

-

```
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: [apache, centos, httpd, web]
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"

- name: Configure Database Servers
  hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      tags: [centos, db, mariadb]
      dnf:
        name: mariadb-server
        state: latest
```

```
    when: ansible_distribution == "CentOS"

  - name: MariaDB - Restarting / Enabling
    tags: [db, mariadb]
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb package (Ubuntu)
    tags: [db, mariadb, ubuntu]
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"

- name: Configure File Servers
  hosts: file_servers
  become: true
  tasks:
    - name: install samba package
      tags: [samba, file]
      package:
        name: samba
        state: latest
```

To orchestrate multiple roles in a single playbook for different server groups.
Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

ansible-playbook –ask-become-pass roles.yaml

```
PLAY RECAP *********************************************************************************
centos1                    : ok=5    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
```

-

ansible-playbook –tags web –ask-become-pass roles.yaml

```
PLAY RECAP *********************************************************************************
centos1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
ubuntu1                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu2                    : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
```

-

ansible-playbook –tags db –ask-become-pass roles.yaml

```
PLAY RECAP *********************************************************************************
centos1                    : ok=5    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
ubuntu1                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
ubuntu2                    : ok=3    changed=0    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
```

-

Reflections:

1. What is the importance of putting our remote servers into groups?
   - Put servers into groups so you can apply the right settings to the right machines. You can also install packages and run jobs across several hosts at the same time, as long as those hosts share the same job, like web or database.

2. What is the importance of tags in playbooks?
   - Use tags to pick only the steps you want. This lets you run a part of a play without doing the full playbook. It can make checks faster and make upkeep easier.

3. Why do you think some services need to be managed automatically in playbooks?
   - Automate how services are handled. Set it so key daemons, such as Apache or MariaDB, come up when the system boots. Also make sure they reload when the config files are updated.

Conclusions/Learnings:

Putting target servers into clear groups based on what they do makes it easier to handle multi tier setups. This works across Ubuntu and CentOS too. You can also split duties into small modules, so changes stay simpler to track.

Using task tags in a careful way helps you run only what you need. At the same time, automated service handling

keeps key background jobs up. Those jobs restart when they should, and they keep working without you doing manual steps.

Github Link: https://github.com/mo-na-min/CPE232_Duenas

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.